Comparison of C++ and Java

Pass parameters to base class

C++

```
class Pet {
    public:
    // Constructors, Destructors
    Pet () : weight(1), food("Pet Chow") {}
    Pet(int w) : weight(w), food("Pet Chow") {}
    Pet(int w, string f) : weight(w), food(f) {}
    ~Pet() {}
}
class Rat: public Pet {
    public:
    Rat() {}
    Rat(int w) : Pet(w) {}
    Rat(int w, string f) : Pet(w,f) {}
    ~Rat() {}
}
```

Java

```
class Pet {
    public:
    // Constructors, Destructors
    Pet () {}
    Pet(int w) {}
    Pet(int w, string f) {}
    ~Pet() {}
}
class Rat: public Pet {
    public:
    Rat() {}
    Rat(int w) {
        super(w) ;
        .......
    }
    Rat(int w, string f) {
        super(w,f);
        ......
    }
    ~Rat() {}
}
```

Array

C++

```
class Pet {
 };


Usage
Pet allpets[100] ;
// declare 100 objects,
   memory space is
   allcated. Constructor
   is called 100 times
```

Java ++

```
class Pet {
 }
Usage
Pet allpets[100] ;
// declare 100 reference,
   no objects are allocated
---- somewhere -----
for (i=0; i<100; i++) {
   allpets[i] = new Pet();
}
```

Parameter Passing to methods

C++

For all primitive type or class type, they can be passed by

- pass by value
- pass by address
- pass by reference

Java

Primitive Types (int, char, float, double…)

- Only pass by value

Class types (a.k.a., objects)

- only pass by reference

Object allocation

C++ (for primitive types or any class types)

- Global variable (in data segment)
- Local variable (in stack)
- Heap variable

Java

- Global variable : can be approximated via static inside a class
- Local variable (for primitive types only)
- Heap variable (for any class types or primitive types)

Abstract class

C++

```
Class human {
      int weight ;
      int height ;
Public:
virtual void walk()=0;
virtual void speak()=0;
}
```

C++

```
public abstract Class human {
      int weight ;
      int height ;
 public abstract void walk() ;
Public  abstract speak();
}
```

Inheritance

- **C++**
- Pass parameter to constructor from subclass to base class
  - Use initialization member list

- **Java**

In the constructor, if you use **super()**, it must be the very first code, and you can not access any **this**. xxx variables or methods to compute its parameters.

```
// using super in a constructor
public MyClass ( Thing thing, Color color )
   {
   // call the original constructor
   super( thing );
   this.color = color;
   }

// using super in a method
public void myMethod ( Font font )
   {
   // use the original myMethod.
   super.myMethod ( font );

   // use the original otherMethod.
   super.otherMethod ( font );
   }
```